



UTM
UNIVERSITI TEKNOLOGI MALAYSIA

SECR2043 Operating System
Group Project

**Free Space Management by Using
Cloud Storage Scenario**

Group Members:

NAME	MATRIC NUMBER
Ain Nurnabila binti Mohd Azhar	A23CS0207
Syarifah Dania binti Syed Abu Bakar	A23CS0183
Damiya Aina binti Basir Abd Shammad	A23CS0220

Abstract

Free space management is very crucial in operating systems to efficiently allocate and also reclaim the disk space. This project implements four main techniques which are Bitmap, Linked List, Grouping, Counting via the context of cloud storage systems for instance Google Drive and also Dropbox. As what has been implemented in Python, this implementation models on how cloud servers track available locks when users upload or delete unused files. This project's goal is to help students to understand the design choices behind free space tracking and visualize how each mentioned method handles allocation and also fragmentation. This simulation explains practical, interactive and useful tool to strengthen basic knowledge of file system internals.

1. Introduction

To store files and guarantee smooth operation, modern operating systems need to effectively manage disk space. The OS monitors unused disk blocks using a process called free space management. File systems may become unstable or inefficient if improperly managed.

Free space can be managed in a variety of ways, each with choices in difficulty, speed, and usage of memory. The four most popular methods which are Bitmaps, Linked Lists, Grouping, and Counting are the subject of this project's models and comparison. In the objective of making OS ideas simpler and interesting, this simulation assists in visualizing how these methods work in real time.

2. Problem Statement

The problem of successfully handling free disk space becomes increasingly important as cloud storage platforms expand in scale. Disk space might be wasted or divided without appropriate free space tracking, which impacts storage availability and performance.

Although these mechanisms are vital, students frequently disregard them or have a poor understanding of them. The objective of this study is to simulate how cloud storage providers could allocate and emit space during interaction with users using various free space management strategies. Through an intriguing design, the simulation combines theoretical ideas with practical applications.

3. Related Works

Operating systems use a number of free space management methods, each of that is selected according to the requirements of the system. Bitmaps are used in ext2

meanwhile linked allocation is used in FAT. These approaches are covered separately in videos uploaded to YouTube and on academic websites such as GeeksforGeeks.

Few implementations, however, connect the four methods or link them to services offered by the cloud. These methods are combined in this project to generate an integrated simulation tool. The simulation gives students a feasible, practical reference that enhances understanding by connecting the functionality to cloud storage actions.

4. Methodology

The project focuses on four free space management techniques:

4.1 Project Management Flowchart

The project is based on six core stages:

1. Planning: Defined simulation scope (cloud storage analogy, free space techniques)
2. Research: Reviewed Operating Systems concepts and cloud storage architecture
3. Design: Design flowchart and function-based system architecture
4. Implementation: Coded the simulation by using Python
5. Testing: Ran the simulation with various block operations
6. Documentation: Compiled final results and wrote the report

4.2 Hardware and Software Requirements

- Hardware: Standard laptop or PC
- Software: Python, Jupyter Notebook Anaconda Navigator

4.3 Task Distribution

Member	Task Distribution
Ain Nurnabila binti Mohd Azhar	Flowchart, Bitmap & Grouping Code, Report Writing, Testing
Syarifah Dania binti Syed Abu Bakar	Flowchart, Bitmap & Grouping Code, Literature Review, Documentation, Testing
Damiya Aina binti Basir Abd Shammad	Final Testing, Code Debugging, Report Formatting, Video Presentation, Testing

5. Design & Implementation

The simulation is organized into four main classes, each representing a free space management technique. Key features:

Allocation/Deallocation: Users can specify the number of blocks to allocate or free.

Visualization: The state of the disk is displayed using color-coded bars (matplotlib), making it easy to distinguish between free and allocated blocks.

Comparison: Users can observe and compare the behavior of each method under different scenarios

5.1 System Flowchart

Start: The process begins.

User requests operation: The user chooses to allocate or free blocks.

Operation type?: Branches to allocation or deallocation logic.

Allocation:

Sets up method-specific variables.

Loops through the free space structure (bitmap array, linked list, group list, or runs).

Checks if a suitable space is found.

If found, marks blocks as allocated and returns the starting index.

If not found, continues looping; if the end is reached, returns failure.

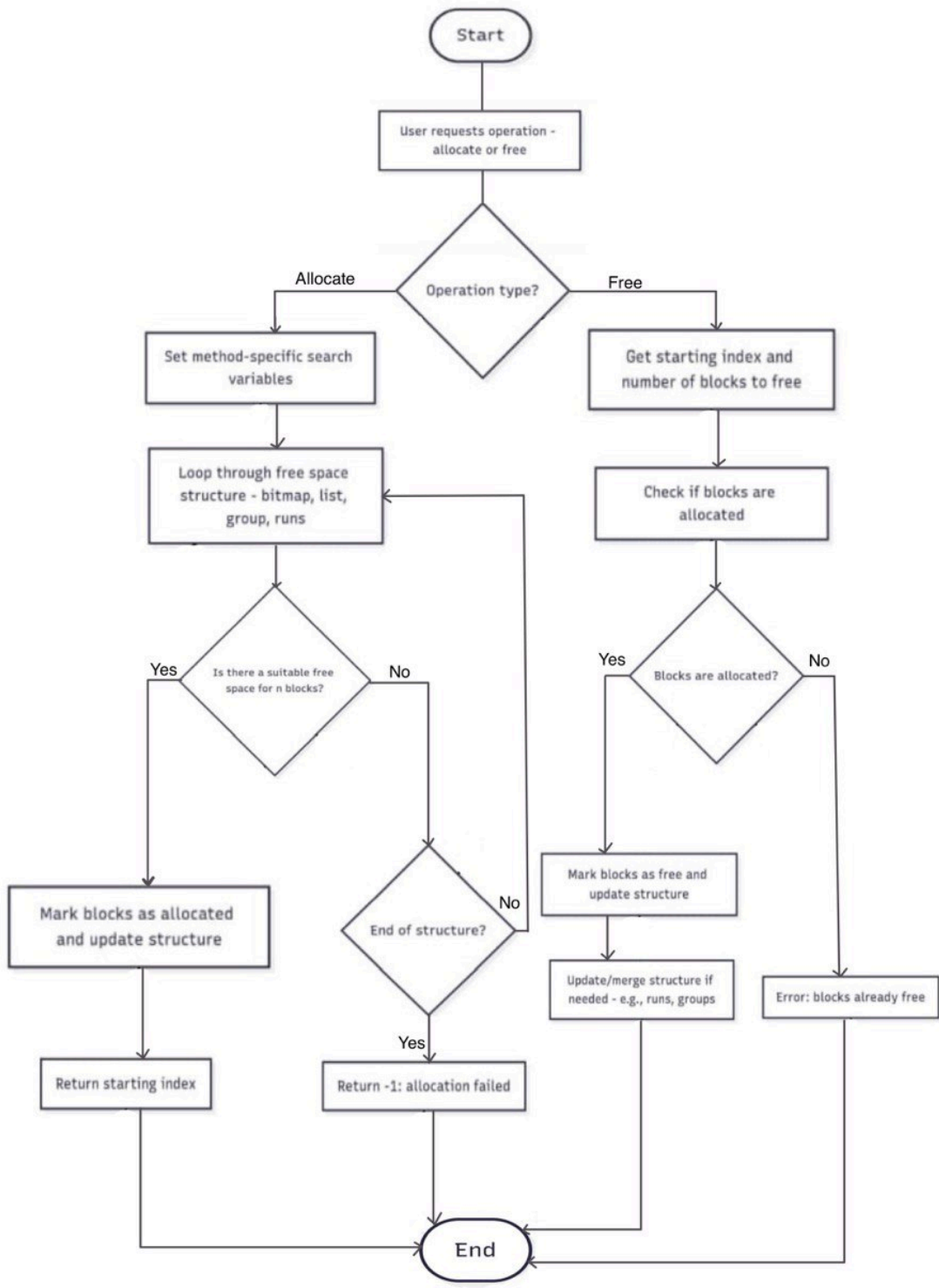
Deallocation:

Gets the starting index and number of blocks to free.

Checks if the blocks are currently allocated.

If yes, marks them as free and updates/merges the structure if needed (e.g., merging runs in Counting).

If not, returns an error.



5.2 Code Implementation

5.2.1 Bitmap Implementation

```
: import matplotlib.pyplot as plt
import numpy as np

: class Bitmap:
    def __init__(self, size):
        self.size = size
        self.bitmap = [0] * size

    def allocate(self, n):
        for i in range(self.size - n + 1):
            if all(self.bitmap[j] == 0 for j in range(i, i + n)):
                for j in range(i, i + n):
                    self.bitmap[j] = 1
                return i
        return -1

    def free(self, start, n):
        for i in range(start, start + n):
            self.bitmap[i] = 0

    def display(self):
        plt.figure(figsize=(8, 1))
        plt.title("Bitmap Free Space")
        plt.imshow([self.bitmap], cmap='Greens', aspect='auto')
        plt.yticks([])
        plt.xticks(range(self.size))
        plt.show()
```

5.2.2 Linked List Implementation

```
class LinkedListNode:
    def __init__(self, index):
        self.index = index
        self.next = None

class LinkedList:
    def __init__(self, size):
        self.size = size
        self.blocks = [0] * size
        self.head = None
        self._init_list()

    def _init_list(self):
        prev = None
        for i in range(self.size):
            node = LinkedListNode(i)
            if prev:
                prev.next = node
            else:
                self.head = node
            prev = node
```

```
    def allocate(self, n):
        prev = None
        curr = self.head
        count = 0
        start = None
        nodes = []
        while curr:
            if self.blocks[curr.index] == 0:
                if count == 0:
                    start = curr
                count += 1
                nodes.append(curr)
                if count == n:
                    for node in nodes:
                        self.blocks[node.index] = 1
                    return start.index
            else:
                count = 0
                nodes = []
            prev = curr
            curr = curr.next
        return -1

    def free(self, start, n):
        for i in range(start, start + n):
            self.blocks[i] = 0
```

```
def display(self):
    plt.figure(figsize=(8, 1))
    plt.title("Linked List Free Space")
    plt.imshow([self.blocks], cmap='Blues', aspect='auto')
    plt.yticks([])
    plt.xticks(range(self.size))
    plt.show()
```

5.2.3 Grouping Implementation

```
class Grouping:
    def __init__(self, size, group_size=4):
        self.size = size
        self.group_size = group_size
        self.blocks = [0] * size
        self.groups = [list(range(i, min(i + group_size, size))) for i in range(0, size, group_size)]

    def allocate(self, n):
        for group in self.groups:
            free_blocks = [i for i in group if self.blocks[i] == 0]
            if len(free_blocks) >= n:
                for i in free_blocks[:n]:
                    self.blocks[i] = 1
                return free_blocks[0]
        return -1

    def free(self, start, n):
        for i in range(start, start + n):
            self.blocks[i] = 0

    def display(self):
        plt.figure(figsize=(8, 1))
        plt.title("Grouping Free Space")
        plt.imshow([self.blocks], cmap='Oranges', aspect='auto')
        plt.yticks([])
        plt.xticks(range(self.size))
        plt.show()
```

5.2.4 Counting Implementation

```
class Counting:
    def __init__(self, size):
        self.size = size
        self.blocks = [0] * size
        self.runs = [(0, size)] # (start, length)

    def allocate(self, n):
        for idx, (start, length) in enumerate(self.runs):
            if length >= n:
                for i in range(start, start + n):
                    self.blocks[i] = 1
                if length == n:
                    self.runs.pop(idx)
            else:
                self.runs[idx] = (start + n, length - n)
                return start
        return -1
```

```
    def free(self, start, n):
        for i in range(start, start + n):
            self.blocks[i] = 0
        # Merge runs
        self.runs.append((start, n))
        self.runs = sorted(self.runs)
        merged = []
        for s, l in self.runs:
            if not merged or merged[-1][0] + merged[-1][1] < s:
                merged.append((s, l))
            else:
                prev_s, prev_l = merged.pop()
                merged.append((prev_s, prev_l + l))
        self.runs = merged

    def display(self):
        plt.figure(figsize=(8, 1))
        plt.title("Counting Free Space")
        plt.imshow([self.blocks], cmap='Purples', aspect='auto')
        plt.yticks([])
        plt.xticks(range(self.size))
        plt.show()
```

6. Results & Discussion

The simulation of free space management techniques, Bitmap, Linked List, Grouping and Counting provides valuable insights into their operational characteristics, strengths and weaknesses. By running allocation and deallocation scenarios and visualizing the state of disk blocks, we can compare their efficiency, memory usage, and suitability for different file system environments.

i) Bitmap

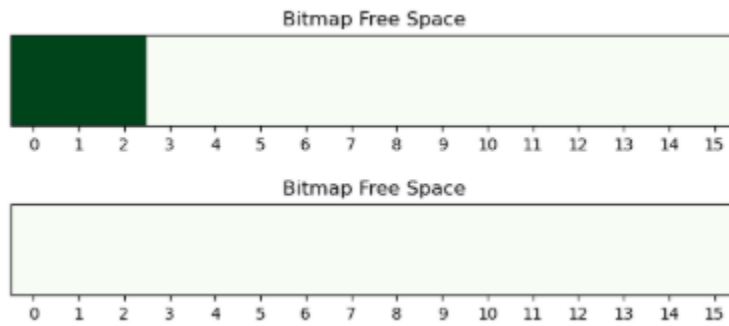
Pros:

- Fast Random Access: The bitmap method allows the operating system to quickly check the status of any block by directly indexing into the bitmap array. This makes it highly efficient for random allocation and deallocation requests.
- Simplicity: The implementation is straightforward, as each block is represented by a single bit, making the logic easy to follow and debug.

Cons:

- High Memory Usage for Large Disks: For very large disks, the bitmap array itself can consume significant memory, as one bit is required for every block on the disk. For example, a disk with millions of blocks will require a bitmap with millions of bits, which can add up to several megabytes of RAM just for tracking free space.
- Fragmentation Handling: While the bitmap can quickly find free blocks, it may not always find large contiguous spaces efficiently, especially as the disk becomes fragmented.

Visualization:



The bitmap visualization clearly shows which blocks are free (e.g., green) and which are allocated (e.g., red). Fragmentation is visible as scattered free blocks among allocated ones.

ii) Linked List

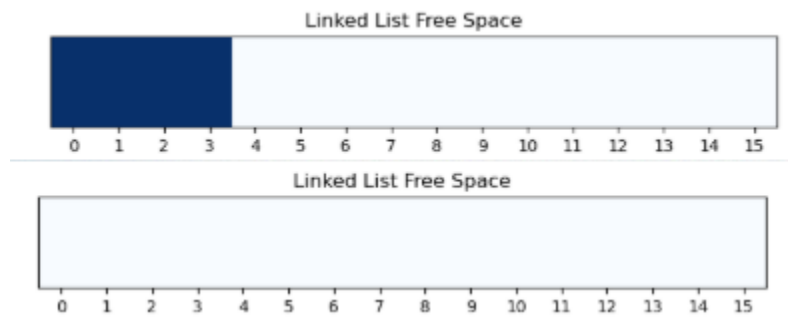
Pros:

- Simple Structure: The linked list method is conceptually simple, with each free block pointing to the next. This makes it easy to traverse and manage.
- Efficient for Sequential Allocation: When files are created and deleted in a sequential manner, the linked list can efficiently allocate and free blocks without much overhead.

Cons:

- Slower Random Access: To find a specific free block or a set of contiguous free blocks, the system must traverse the list, which can be slow for large or highly fragmented disks.
- Pointer Overhead: Each free block must store a pointer to the next, which adds overhead compared to the bitmap method, especially if the pointer size is significant relative to the block size.

Visualization:



The linked list visualization helps users see the chain of free blocks. Fragmentation appears as many short chains or isolated free blocks, making it easy to understand the impact of allocation patterns.

iii) Grouping

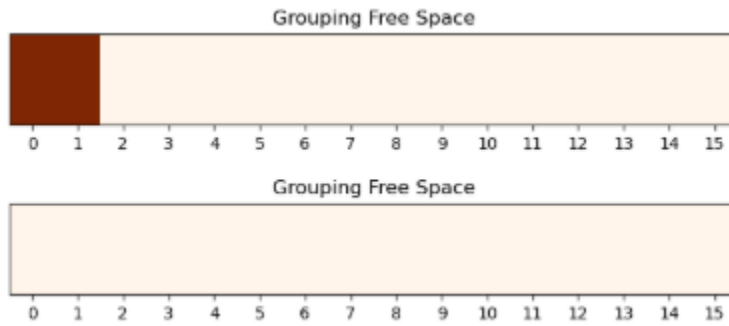
Pros:

- Reduces Traversal Overhead: By grouping free block addresses together, the system can quickly find and allocate multiple free blocks at once, reducing the time spent searching for free space.
- Efficient for Grouped Free Blocks: This method is particularly effective when free blocks tend to be clustered together, as it can allocate large groups with minimal effort.

Cons:

- Complex Implementation: Managing groups and updating them as blocks are allocated or freed adds complexity to the implementation.
- Less Flexible for Random Allocation: If free blocks are scattered rather than grouped, the grouping method may not be able to efficiently allocate the requested space, leading to allocation failures or increased search times.

Visualization:



Grouping is visualized by highlighting clusters of free blocks. The effectiveness of this method is apparent when free space is naturally grouped, but its limitations become clear in highly fragmented scenarios.

iv) Counting

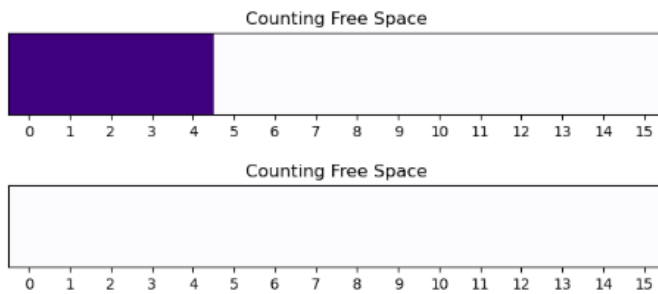
Pros:

- Space-Efficient for Large Contiguous Free Areas: The counting method records only the starting address and the number of contiguous free blocks, making it very efficient when there are large, unbroken stretches of free space.
- Low Memory Overhead: Fewer records are needed to track free space, especially when fragmentation is low.

Cons:

- Less Effective with Fragmented Free Space: As the disk becomes fragmented, the number of records increases, and the method loses its space and performance advantages. Allocation may also become slower as the system must search through many small runs to find a suitable space.

Visualization:



The counting method's visualization shows runs of free blocks, making it easy to see how fragmentation increases the number of runs and reduces the method's efficiency.

Performance and Memory Usage

- Bitmap is the fastest for random access because it allows direct indexing, making it ideal for workloads with frequent, unpredictable allocations and deallocations. However, its memory usage grows linearly with disk size.
- List is efficient for sequential operations but suffers in random access scenarios due to the need to traverse the list.
- Grouping offers a balance, excelling when free space is naturally clustered but struggling with scattered free blocks.
- Counting is the most space-efficient when free space is contiguous, as it minimizes the metadata needed to track free blocks. However, its performance and memory efficiency degrade as fragmentation increases.

Visualization Benefits

The simulation's visualizations are crucial for understanding:

- Fragmentation: Users can see how different allocation and deallocation patterns affect the distribution of free space.
- Efficiency: The ease or difficulty of finding free space is visually apparent, especially as the disk fills up or becomes fragmented.

- Method Suitability: By experimenting with different scenarios, users can see which method performs best under various conditions, reinforcing theoretical knowledge with practical experience.

In conclusion, the simulation not only demonstrates the theoretical differences between free space management techniques but also provides practical, visual evidence of their strengths and weaknesses, helping users make informed decisions about which method to use in different file system scenarios.

7. Future Work & Conclusion

While the current simulation effectively models and visualizes four free space management techniques, there is substantial potential for further development to enhance user experience, realism, and educational value. Below are several areas identified for future enhancement:

7.1 Future Work

Future Enhancements:

i) Developing a graphical user interface (GUI) for easier interaction.

A graphical interface would allow users to interact with the simulation more intuitively, without needing to modify code or run functions manually. Features such as dropdown menus to select techniques, sliders to set block sizes, and real-time visual updates of disk status could provide a more engaging learning experience.

ii) Simulating files of varying sizes and types.

In real-world file systems, files differ in size and structure. The current simulation assumes fixed-size blocks for allocation. Future versions could include:

Files with variable sizes (e.g., small text files vs. large multimedia files)

Support for file attributes such as file type, creation time, and modification history

Allocation policies that mimic real systems (e.g., first-fit, best-fit, next-fit)
This enhancement would allow for more realistic testing of each technique's performance and suitability in practical scenarios.

iii) Adding visualization for fragmentation and defragmentation.

iv) Measuring and displaying performance metrics (e.g., allocation time, memory usage).

7.2 Conclusion

This project shows basic Java free space management strategies. The simulation offers both practical grasp and helpful understanding. It successfully models and visualizes four key free space management techniques, providing a practical tool for understanding their operation and trade-offs.

Free space management is vital in operating system design. Through this project, we have modeled and visualized four major strategies, allowing users to interactively learn their behavior and limitations. The simulation bridges the gap between textbook knowledge and real-world application. Each method has its trade-offs, and understanding them is key to designing efficient file systems.

8. Acknowledgement

We would like to express our sincere gratitude to all individuals and parties who have supported and guided us throughout the development of this project entitled Free Space Management in Operating Systems.

First and foremost, we extend our heartfelt appreciation to Dr Danlami Gabi and Dr Zafran, our course instructors for SECR2043 Operating Systems, for their valuable guidance, constructive

feedback, and continuous encouragement throughout the semester. Their dedication to teaching and commitment to student learning inspired us to explore the topic in depth and approach the project with confidence.

We would also like to thank the Faculty of Computing, Universiti Teknologi Malaysia (UTM) for providing us with the necessary resources and facilities to carry out this project. The learning environment and support system have played a significant role in the success of this assignment.

Our sincere thanks also go to our classmates and peers, whose insightful discussions and suggestions helped us refine our understanding and approach. Their shared experiences and collaboration have made this project journey more meaningful.

Special appreciation goes to each group member for their outstanding teamwork, mutual respect, and unwavering dedication. The contribution of every member, Ain Nurnabila binti Mohd Azhar, Syarifah Dania binti Syed Abu Bakar and Damiya Aina binti Basir Abd Shammad was crucial in ensuring the timely and successful completion of this project. Through effective communication and shared responsibility, we were able to overcome challenges and learn from one another.

Lastly, we acknowledge the authors of textbooks, online tutorials, educational platforms such as GeeksforGeeks and YouTube creators, whose open-access content significantly enhanced our understanding of the concepts and techniques used in this project.

This project would not have been possible without the combined efforts, support, and encouragement from all those mentioned above. We are truly grateful.

9. References

McAuliffe, M. L., Carey, M. J., & Solomon, M. H. (1996). Towards effective and efficient free space management. *ACM SIGMOD Record*, 25(2), 389–400.

<https://doi.org/10.1145/235968.233355>

Forell, T., Milojicic, D., & Talwar, V. (2011). *Cloud Management: Challenges and Opportunities*. IEEE Xplore. <https://doi.org/10.1109/IPDPS.2011.233>

Sankaran Sivathanu, Liu, L., Yiduo, M., & Pu, X. (2010). Storage Management in Virtualized Cloud Environment. *CiteSeer X (the Pennsylvania State University)*, 204–211. <https://doi.org/10.1109/cloud.2010.27>

Fusco, F., Vlachos, M., & Marc Ph. Stoecklin. (2011). Real-time creation of bitmap indexes on streaming network data. *The VLDB Journal*, 21(3), 287–307.
<https://doi.org/10.1007/s00778-011-0242-x>

Chin, Y. H. (1978). An Analysis of “Distributed Free Space” in an Operating and Data Management Systems Environment. *IEEE Transactions on Software Engineering*, SE-4(5), 436–440. <https://doi.org/10.1109/tse.1978.233863>

Appendix

Youtube Link: <https://youtu.be/J9wObb81fW0>

